

总体评价



刘光悦 按时通关

学号：U202414665

分班：24级大数据01班

分组：未分组

截止前完成关卡：4/4

最新完成关卡：4/4

完成效率：--

课堂最高完成效率：--

通关时间	计时规则 ?	实训总耗时 ?	评测次数	查重扣分	补交扣分	最终成绩	总评
2026-06-03 16:25	页面停留时长	5分 50秒	4	--	--	24.0/24.0	优秀

阶段成绩

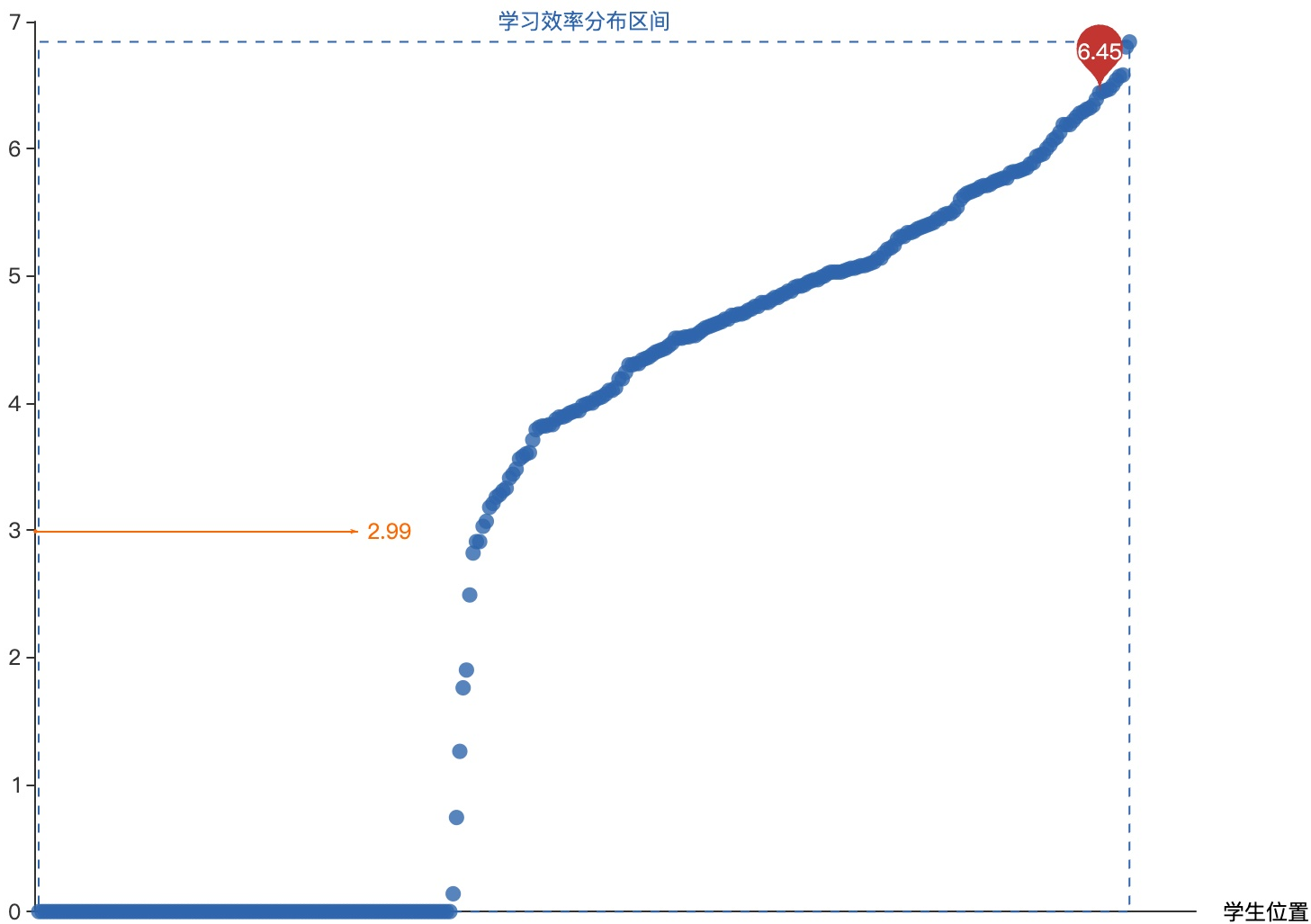
关卡	任务名称	开启时间	代码修改行数	关卡要求	评测次数	完成时间	实训耗时	是否查看答案	经验值	关卡得分	调分
1	磁盘存储管理器(Disk Manager)	2026-06-03 16:19	69	必做	1	2026-06-03 16:22	3分 17秒	否	1000/1000	6.00/6	6.00
2	缓冲池替换策略(Page Replacer)	2026-06-03 16:22	24	必做	1	2026-06-03 16:23	44秒	否	1000/1000	6.00/6	6.00
3	缓冲池管理器(Buffer Pool Manager)	2026-06-03 16:23	82	必做	1	2026-06-03 16:24	45秒	否	1000/1000	6.00/6	6.00
4	记录管理器(Record Manager)	2026-06-03 16:24	--	必做	1	2026-06-03 16:25	1分 4秒	否	700/700	6.00/6	6.00

图形统计

学习效率

学习效率=log(实训总得分/实训总耗时)

姓名 刘光悦
学号 U202414665



学习效率 6.45
当前排名 9

1个小圆点代表1个学生的学习效率
红色水滴表示当前学生的学习效率值
小圆点越高，学习效率越高
橙色横向箭头：中位值，表示整个课堂学生表现的平均值

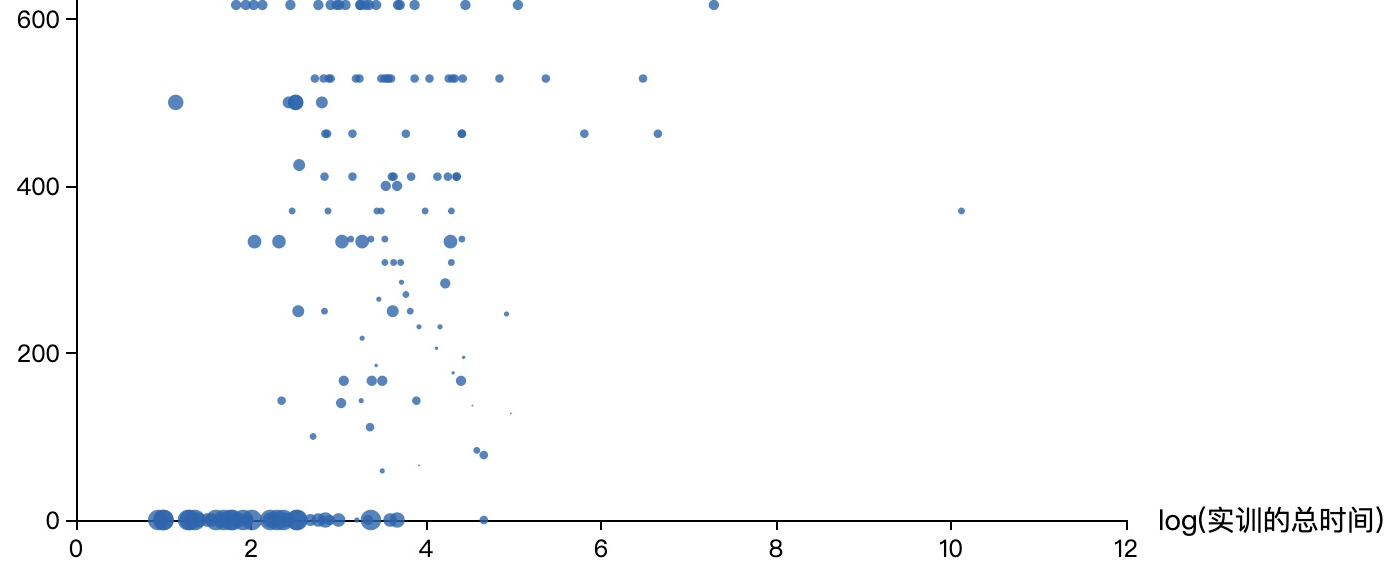
能力值

能力值（实训获得经验值/实训评测次数）



姓名 刘光悦
学号 U202414665
能力 925

1个小圆点代表1个学生的能力值
红色水滴表示当前学生的能力值
小圆圈越大，评测次数越少



实训详情



学员代码

代码修改行数 69

rmdb/src/storage/disk_man

复制代码

```
1  #include "storage/disk_manager.h"
2
3  #include <assert.h>    // for assert
4  #include <string.h>    // for memset
5  #include <sys/stat.h>  // for stat
6  #include <unistd.h>    // for lseek
7
8  #include "defs.h"
9
10 DiskManager::DiskManager() {
11     memset(fd2pageno_, 0, MAX_FD * (sizeof(std::atomic<page_id_t>) / sizeof(char)));
12 }
13
14 /**
15  * @brief Write the contents of the specified page into disk file
16  *
17  * @description: 将数据写入文件的指定磁盘页面中
```

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	评测通过	134.2	724.61	2026-06-03 16:22	查看代码

实验结果

提交次数	提交时间	评分	操作
------	------	----	----



暂无数据

暂无评价~



学员代码

代码修改行数 24

rmdb/src/replacer/lru_repla

```
1 #include "lru_replacer.h"
2
3 LRURemplacer::LRURemplacer(size_t num_pages) { max_size_ = num_pages; }
4
5 LRURemplacer::~LRURemplacer() = default;
6
7 /**
8  * @brief 使用LRU策略删除一个victim frame, 这个函数能得到frame_id
9  * @param[out] frame_id id of frame that was removed, nullptr if no victim was found
10  * @return true if a victim frame was found, false otherwise
11  */
12 bool LRURemplacer::Victim(frame_id_t *frame_id) {
13     // C++17 std::scoped_lock
14     // 它能够避免死锁发生, 其构造函数能够自动进行上锁操作, 析构函数会对互斥量进行解锁操作, 保证线程安全。
15     std::scoped_lock lock{latch_};
16
17     // Todo:
```

复制代码

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	评测通过	29.86	339.58	2026-06-03 16:23	查看代码

实验结果

提交次数	提交时间	评分	操作
------	------	----	----



暂无数据

暂无评价~



学员代码

代码修改行数 82

rmdb/src/storage/buffer_pc

复制代码

```
1  #include "buffer_pool_manager.h"
2
3  /**
4   * @brief 从free_list或replacer中得到可淘汰帧页的 *frame_id
5   * @param frame_id 帧页id指针,返回成功找到的可替换帧id
6   * @return true: 可替换帧查找成功 , false: 可替换帧查找失败
7   */
8  bool BufferPoolManager::FindVictimPage(frame_id_t *frame_id) {
9      // Todo:
10     // 1 使用BufferPoolManager::free_list_判断缓冲池是否已满需要淘汰页面
11     // 1.1 未满足获得frame
12     // 1.2 已满使用lru_replacer中的方法选择淘汰页面
13
14     if (this->free_list_.empty()) {
15         if (!this->replacer_->Victim(frame_id)) { // 空闲帧不足,调用LRU淘汰
16             return false; // 淘汰失败
17         }
```

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	评测通过	28.33	597.93	2026-06-03 16:24	查看代码

实验结果

提交次数	提交时间	评分	操作
------	------	----	----



暂无数据

暂无评价~



学员代码 ?

rmdb/src/record/rm_file_ha

```
1  #include "rm_file_handle.h"
2
3  /**
4   * @brief 由Rid得到指向RmRecord的指针
5   *
6   * @param rid 指定记录所在的位置
7   * @return std::unique_ptr<RmRecord>
8   */
9  std::unique_ptr<RmRecord> RmFileHandle::get_record(const Rid &rid, Context *context) const {
10     // Todo:
11     // 1. 获取指定记录所在的page handle
12     // 2. 初始化一个指向RmRecord的指针 (赋值其内部的数据和size)
13     RmPageHandle tmp_page_handle = fetch_page_handle(rid.page_no);
14     int size_ = tmp_page_handle.file_hdr->record_size;
15     char *data_ = tmp_page_handle.get_slot(rid.slot_no);
16     //printf("在get_record里面为%s\n",data_);
17     std::unique_ptr<RmRecord> record_ptr(new RmRecord(size_,data_));
```

复制代码

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	评测通过	13.1	315.14	2026-06-03 16:25	查看代码

实验结果

提交次数	提交时间	评分	操作
------	------	----	----



暂无数据

暂无评价~

